Программная инженерия. Лекция 2
Unified Process

Михаил Пожидаев

12 февраля 2026 г.

Итерации и спринты

Можно повторить, если прошло неудачно

Итерации и спринты (термин из мира гибких методологий) позволяют разбивать процесс разработки на управляемые отрезки времени, в течение которых команда разработчиков работает над определённым набором функций или задач. Разбиение процесса разработки на короткие итерации позволяет оперативно выявлять и устранять потенциальные проблемы, снижая риск неудачи проекта.

Только в плюсе!

Инкремент означает добавление новых функций или улучшений к ПО в результате каждой итерации или спринта. Это позволяет постепенно наращивать функциональность продукта. Отсутствие инкремента приводит к растягиванию процесса разработки или к накоплению технических долгов.

Ключевой элемент цикличности! Итерации и спринты являются ключевой возможностью предпринять несколько попыток к решению некоторой задачи без перевода проекта в критическое состояние при неудаче.

MoSCoW и UpSide

Ключевые понятия анализа требований

- *Must Have:* критичное требование;
- *Should Have:* приоритетное требование;
- *Could Have:* желательное требование;
- *Won't Have: this time*: перспективное требование.

Upside

В управлении проектами представляет собой потенциал положительного отклонения от запланированных показателей проекта. Этот термин отражает возможности для достижения дополнительных выгод, оптимизации ресурсов и повышения эффективности выполнения задач в рамках проектного цикла.

Артефакты

Примеры и типы

Артефакт — это общее название для любых видов информации, создаваемой, изменяемой или используемой сотрудниками при создании системы. Примерами артефактов могут служить:

- диаграммы UML и связанный с ними текст;
- наброски и прототипы пользовательских интерфейсов, компоненты;
- планы тестирования и тестовые примеры.

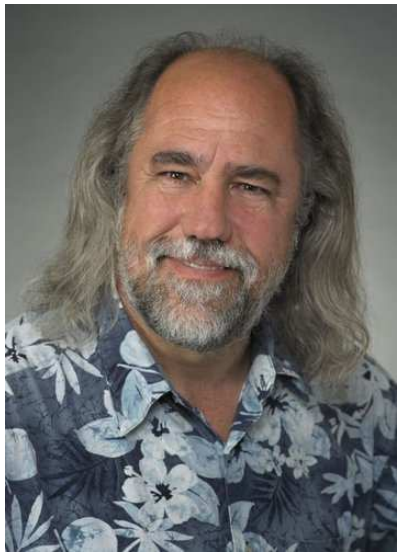
Можно выделить два основных типа артефактов:

- технические артефакты;
- артефакты управления.

Создатели UP и UML

Unified Process и Unified Modelling Language

UP и UML были предложены группой специалистов, включая Гради Буча, Ивара Якобсона и Джеймса Румбо, на заре развития объектно-ориентированного анализа и проектирования. Это произошло в эпоху, когда возникла потребность в создании целостной методологии для разработки сложных программных систем.



Гради Буч, Ивар Якобсон и Джеймс Рамбо

Unified Process

Назначение, ключевые особенности и требования

Unified Process (UP) — это итеративная методология разработки программного обеспечения, предназначенная для создания высококачественных продуктов с предсказуемыми характеристиками. UP обеспечивает структурированный подход к разработке, позволяя командам эффективно управлять процессом создания сложных систем. Процесс разработки — это сумма различных видов деятельности, необходимых для преобразования требований заказчика в программную систему.

Функции UP:

- обеспечивает руководство деятельностью команды;
- управляет задачами отдельного разработчика и команды в целом;
- указывает очередность разработки артефактов;
- предоставляет KPI для оценки параметров процесса разработки и функционирования проекта.

Фазы

Четыре принципиальные вехи развития проекта

Унифицированный процесс предусматривает четыре фазы, каждая из которых заканчивается вехой проекта:

- **анализа (inception):** анализ требований и построение функционального облика;
- **проектирования (elaboration):** построение архитектуры;
- **реализации (construction):** написание кода и тестирование;
- **внедрения (transition):** передача продукта заказчику и обеспечение его функционирования.

Нельзя!

Возврат к предыдущей фазе запрещён!

жизненный цикл

Цикличность разработки в традициях UP

UP циклически повторяется. Эта последовательность повторений фаз UP представляет собой жизненный цикл системы. При правильно организованной работе одинаковые исходные требования в жизненном цикле должны приводить к одинаковым результатам при его завершении. Каждый цикл завершается поставкой продукта заказчику.

Внимание!

Если проект сталкивается с трудностями, делающими невозможным его реализацию, жизненный цикл всё равно завершается, а результат передаётся заказчику. Заказчик корректирует свои исходные требования, что позволяет повторить жизненный цикл с новыми входными данными. Это приводит к удорожанию проекта, на что заказчик должен дать своё согласие.

Особенности UP

Отличительные черты Unified Process

Нельзя!

1. Модель анализа выработанная на фазе анализа, в принципиальных чертах не подвергается пересмотру вплоть до окончания жизненного цикла.
2. Участники коллектива не могут быть критическими носителями знаний. Все артефакты обязательно должны быть формализованы и отражены на бумаге или в документах:
   - это приводит к дополнительным рабочим усилиям, включая введение в колектив дополнительных сотрудников;
   - любой член коллектива всегда может быть заменён.

Use cases

UP управляется вариантами использования

Вариант использования — это часть функциональности системы, необходимая для получения пользователем значимого для него, ощутимого и измеримого результата.

Периметр системы и мир вокруг:

- usecase **должны быть перечислены все**;
- отсутствующие в usecase требования не могут учитываться при дальнейшем проектировании системы;
- usecase разделяют мир внутри системы и окружающий её мир.

Use case

Диаграмма вариантов использования

Вход в систему

Детализация варианта использования

Не одно и то же

Use case не функция программы!

Так думали раньше, но сейчас уже нельзя!

Считается, что описание функций может ответить на вопрос: «что система делает?».

Следует вспомнить про роль пользователя

Подход вариантов использования добавляет три слова: «для каждого пользователя».

Варианты использования являются основой для написания функциональных тестов.

Фаза анализа

Первая фаза в проекте под управлением UP

На входе!

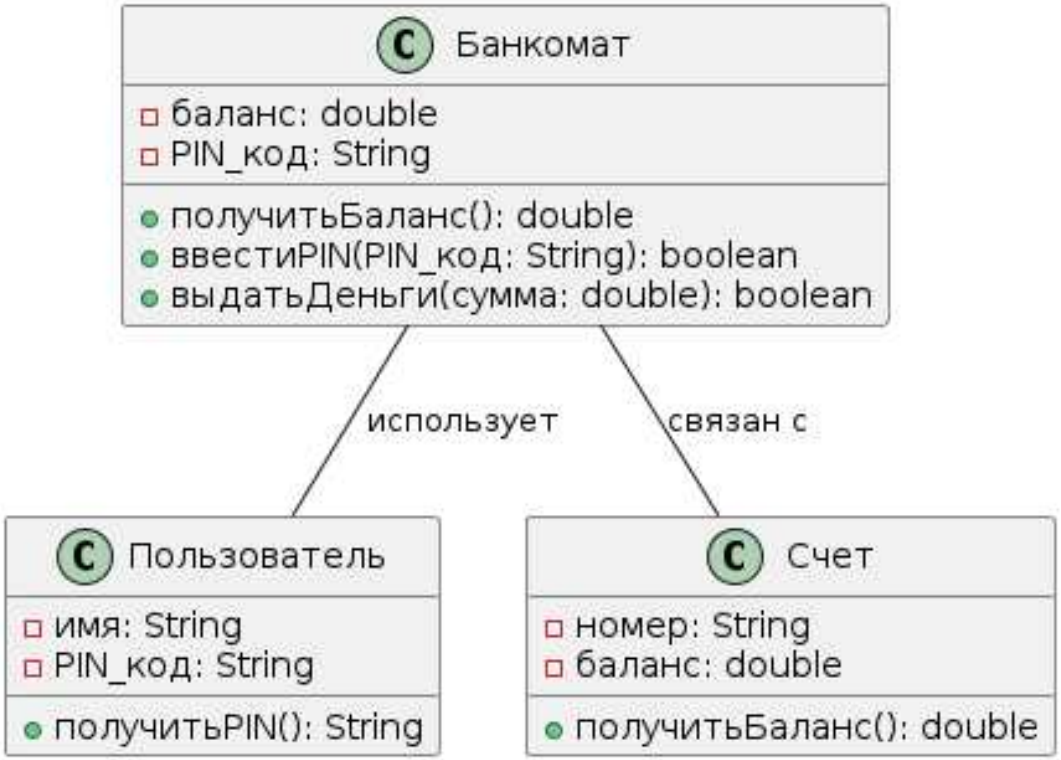
Функциональный запрос заказчика с его готовностью оплатить разработку программного продукта, решающего существующую на стороне заказчика производственную, организационную, досуговую или любую другую проблему.

На выходе модель анализа!

1. Множество определённых вариантов использования.
2. Множество нефункциональных требований.
3. Аналитическая модель классов.
4. Бизнес-план.

Диаграмма классов

Аналитическая модель продукта



Классы

Стереотипы анализа

В модели анализа используется три стереотипа классов :

- **граничный класс:** обеспечивает связь системы с окружающим миром;
- **управляющий класс:** определяет логику принятия решений;
- **класс сущности:** отвечает за представление структур данных и их хранение.

Пример классов

Вариант для банкомата

1. Устройство выдачи и интерфейс ввода/вывода — это граничные классы. Они в основном используются для моделирования взаимодействия между системой и ее актантами.
2. Управление выдачей наличных — управляющий класс. Классы такого типа в основном используются для представления координации, последовательности, взаимодействия и управления другими объектами.
3. Банковский счет — класс сущности;. Такие классы в основном используются для моделирования инкапсуляции.

Назначение класса

Обязанности и роли

Что такое обязанность класса?

Обязанности класса — это просто сумма всех ролей, которые он исполняет во всех реализациях вариантов использования.

Сложив все роли и удалив перекрывающиеся части ролей, мы получим список всех обязанностей и атрибутов класса.

Кто отвечает?

За определение ролей класса отвечают системные аналитики и архитектор.

Подсистемы

Группировка классов

Классы группируются в подсистемы!

Для большой системы, состоящей из сотен или тысяч классов, невозможно реализовать варианты использования, пользуясь только классами. Такая система будет слишком велика для того, чтобы в ней можно было обойтись без группировки высшего порядка.

Подсистема — это осмысленная группировка классов или других подсистем. Подсистема имеет набор интерфейсов, которые обеспечивают ее существование и использование.

Спасибо за внимание!

Всё о курсе: https://marigostra.ru/materials/engineering.html
Канал в Телеграм: https://t.me/MarigostraRu